Pokedex ML

Simon Toft
Khiem Anthony Phan
16.10-26

1. Prosjektet

1.1 Omfang

Vi har et prosjekt der brukeren kan skrive inn de seks kjerne-statistikkene (HP, Attack, Defense, Special Attack, Special Defense og Speed) samt typen(e) til en Pokémon. Basert på disse inndataene analyserer maskinlæringsmodellen mønstre og mønstergjenkjenning i statistikkprofilene for å gi en presis prediksjon på hvilken Pokémon det dreier seg om. Modellen trenes opp på historiske data om Pokémon-egenskaper og benytter en fleksibel teknologistakk for sømløs kjøring og brukerinteraksjon:

- **Streamlit:** Brukes til å bygge et interaktivt og brukervennlig webgrensesnitt der brukeren enkelt kan justere stats og typer.
- **Pandas:** Håndterer innlasting, strukturering og forhåndsbehandling av Pokémon-datasettet.
- **scikit-learn:** Inneholder algoritmene som brukes til å trene maskinlæringsmodellen og utføre klassepredikasjoner.

joblib: Brukes til serialisering, lagring og rask innlasting av den ferdig trente modellen i applikasjonen.

1.2 Metrikker

Siden prosjektet bruker en instansbasert modell (Nearest Neighbors) for å finne hva Pokèmon som matcher jevne kjennetegn, baserer evalueringen seg på vektoravstand framfor tradisjonelle tapsfunksjoner.

- **Avstandsmetrikk:** Vi nyttet eukalidsk avstand på standardiserte verdier (StandardScaler). Skalering var kritisk for at numeriske egenskaper med høye verdier ikke skulle dominere over de binære type-variablene.
- **Likeskapsscore:** For å gjøre resultat intuitivt for sluttbrukeren, konverterte vi avstanden *d* til en normalisert invers likeskapsscore, presentert som en prosentvis sannsynlighetsfordeling over de 3 nærmeste naboene.
- **Verifisering:** Modellen var evaluert ved å laste ned eksakte profiler fra datasettet. I alle testtilfelle returnerte modellen den korrekte arten som førstevalg med 0 i avstand, samt logiske og nærliggende alternativ på 2.- og 3.-plass

2. Data

Vi brukte nettsiden kaggle for å finne datasettet "Pokémon with stats” av Alberto Barradas. Datasettet inneholder ca 800 forskjellige Pokèmons, og kolonnene inneholder

- **Numeriske egenskaper:** HP, Attack, Defense, Sp. Atk, Sp. Def og speed
- **Kategoriske egenskaper:** Navn, Type 1, Type 2, Generasjon og legendary

For pokemons med 2 typer var det en tom verdi, vi måtte derfor erstatte den tomme verdien med “None” i koden.

3. Modellering

Vi valgte **Nearest Neighbor** fra skit-learn som vår algoritme. Siden datasettet inneholder rundt 800 forskjellige Pokemons der hver art bare er representert med en rad, ville en tradisjonell klassifiseringsmodell (som RandomForsest) fungert dårlig til å predikere artnavn direkte. Et avstandsbasert nabosøk er derfor den mest egnet tilnærmingen for dette prosjektet.

Maskinlæringsmodeller har krav på numeriske verdier. Vi nytta derfor **One-Hot Encoding** (pd.get_dummies i train koden) på type 1 og type 2, noe som konverterer elementtyper til binære indikatorvariabler (0 eller 1). Fordi avstandsberekninger er ganske så følsomme for forskjelligheter i måleskalaer, standardiserte vi alle attributt ved hjelp av **StandardScaler**. Dette transformerer hver egenskap til et gjennomsnitt på 0 og en variasjon på 1, og hindrer at store numeriske verdier totalt dominerer over de binære type-kolonnene.

4. Deployment

Modellen blir satt i drift gjennom en webapplikasjon laget med Streamlit og Python. Brukeren kan velge Pokémon-type (Primær og Sekundær) i tillegg til å kunne legge inn verdier for HP, Attack, Defense, Sp. Atk, Sp. Def og Speed. Applikasjonen bruker deretter *NearestNeighbors* til å finne de 3 Pokémonene i datasettet som ligner mest på de valgte verdiene. Resultatet vises i applikasjonen med navn og avstand til søket.

Prediksjonene kan brukes til å finne Pokémon med lignende egenskaper. Dette kan være nyttig for å kunne sammenligne Pokémon eller finne alternativer med omtrent samme statistikk. Systemet finner de nærmeste treffene basert på statistikken, og resultatene skal derfor tolkes som en likhet og ikke som en sikker prediksjon eller sannsynlighet

For å vedlikeholde systemet bør vi kontrollere at applikasjonen fungerer som den skal, at datasett leses inn riktig og at modellen gir relevante resultater. Dersom vi skulle ha oppdatert datasettet med nye Pokémon, så må modellen bygges på nytt.

Videre kan systemet forbedres ved at vi kan legge til flere Pokémon-data, forbedre presentasjonen av resultatene og la brukeren sammenligne flere Pokémon direkte. I tillegg til at vi eventuelt kan legge inn bilder av hver Pokémon som blir søkt opp og eventuell beskrivelse av de ulike Pokémonene.

Demoside

5. Referanser

- Dataset
- GithubRepo